

Prioritizing Bug Issue Reports in GitHub: A Comparative Study of Transformer Models

Sameer Khan

*Department of Computer Science
Bowling Green State University
Bowling Green, Ohio, USA
sameerk@bgsu.edu*

Abbas Heydarnoori

*Department of Computer Science
Bowling Green State University
Bowling Green, Ohio, USA
aheydar@bgsu.edu*

Abstract—Manual prioritization of bug reports in large software repositories like GitHub is time-consuming and error-prone. This study investigates and compares the effectiveness of different transformer-based machine learning models for automating bug issue prioritization. We leverage the GIRT-Data dataset, focusing on Java repositories, and utilize Issue Report Template (IRT) text combined with engineered repository metadata features. We evaluate three approaches: fine-tuning CodeBERT (a model pre-trained on code and text), fine-tuning standard BERT, and few-shot prompt engineering with a GPT-3.5-like model (Airoboros). Our comparative analysis reveals that while a modified CodeBERT configuration (using SMOTE and threshold tuning) achieved the highest recall (1.0000) and F1-score (0.7634) on balanced data, the prompt-engineered GPT-3.5 model attained the highest accuracy (0.7270) on the original unbalanced data. Standard BERT also demonstrated strong, balanced performance (F1-score 0.7586) on the original data without specialized pre-training or data balancing. These findings suggest that general-purpose models like BERT and prompt-engineered LLMs are highly effective alternatives for bug prioritization, potentially outperforming specialized models depending on the evaluation context and specific performance goals (accuracy vs. recall). The choice of model should consider these trade-offs to best suit practical bug triaging workflows.

1. Introduction

The larger the scale and more complex a software project, bug report management and prioritization become more challenging for development teams. As an example, GitHub hosts huge numbers of open-source repositories that generate hundreds or thousands of issues, which makes problematic the identification and resolution of bugs in an effective way. Manual prioritization is often time-consuming, inconsistent, and delayed, potentially affecting the timely resolution of critical bugs.

1.1. Problem Statement

Bug prioritization faces the key challenge of efficiently identifying and sorting the most critical issues out of a

large and continuously growing set of bug reports. If prioritization mechanisms are inefficient or incoherent, critical issues might not get resolved promptly, affecting software quality and user experience. Traditional approaches, such as rule-based systems or manual review processes, often do not scale well with large repositories and may miss important factors like repository activity, report detail, and issue severity. Thus, there is a strong demand for automated and reliable prioritization approaches that can categorize bug reports based on urgency, helping teams allocate resources effectively and respond promptly.

1.2. Objective

This study explores and compares the use of different transformer-based language models – specifically CodeBERT, standard BERT, and a GPT-3.5-like model (Airoboros) – to classify bug reports as high or low-priority within GitHub repositories. CodeBERT is evaluated for its specialized pre-training on programming and natural language data. Standard BERT serves as a general-purpose baseline, while the GPT-3.5-like model explores the potential of large language models via prompt engineering. By fine-tuning CodeBERT and BERT, and prompting Airoboros using a robust dataset of GitHub issues, this research aims to identify effective automated classification models for identifying high-priority issues, enabling development teams to focus on addressing critical bugs efficiently.

1.3. Research Contributions

This paper makes the following contributions to the field of automated bug prioritization:

- **Comparative Analysis of Transformer Models for GitHub Issue Prioritization:** We present a comparative study evaluating CodeBERT, standard BERT, and a GPT-3.5-like model (Airoboros via prompt engineering) for classifying GitHub issue priority. This provides insights into the relative effectiveness of different architectures for this task.

- **Application of Fine-tuning and Prompt Engineering:** We demonstrate methodologies for both fine-tuning (CodeBERT, BERT) and few-shot prompt engineering (Airoboros) applied to bug report classification, incorporating textual and repository meta-data.
- **Feature Engineering from Repository and Issue Data:** To enhance model accuracy, we incorporate features derived from repository activity (e.g., stargazer count, commit frequency, recent updates) and Issue Report Template (IRT) characteristics (e.g., template length, structure).
- **Analysis of Performance Trade-offs:** We analyze the performance of the models using accuracy, precision, recall, and F1-score, highlighting trade-offs. For CodeBERT, we specifically investigate the impact of threshold adjustments and class weighting on the precision-recall balance.

1.4. Paper Structure

The rest of this paper is organized as follows: In Section 2, we provide background information on bug prioritization and the use of transformer models like CodeBERT. Next, Section 3 describes our proposed approach, including dataset preparation, feature engineering, and the configuration for each model tested. Section 4 presents the evaluation setup, research questions, and comparative results, including quantitative and qualitative analyses. In Section 5, we address threats to validity. Section 6 reviews related work in bug prioritization and machine learning applications. **Section 7 discusses the findings from our comparative analysis.** Section 8 discusses the limitations of this study. Finally, Section 9 provides conclusions and suggests directions for future research.

2. Background

2.1. Bug Prioritization in Software Development

In software development, effective bug prioritization is essential for maintaining software quality and ensuring a positive user experience. As software applications grow in complexity, the number of bug reports submitted by users, testers, and developers increases significantly. Without a systematic way to prioritize these issues, critical bugs may be overlooked, leading to unresolved problems that can disrupt user operations or damage the reputation of the software provider. Studies indicate that efficient bug prioritization can help development teams allocate resources better, reduce maintenance costs, and improve overall project productivity [1], [2].

2.2. Challenges in Current Systems

Traditional bug prioritization methods rely heavily on manual processes or basic rule-based systems, which often

fall short in large-scale environments. In manual prioritization, developers or project managers assign priority levels based on their understanding of the issue’s severity and impact. However, this approach is time-consuming, subject to human error, and inconsistent across teams [3]. Rule-based methods, on the other hand, apply predefined criteria to prioritize bugs, which may work for small projects but struggle to capture the nuanced requirements of larger, more complex projects. These systems lack adaptability and can fail to account for dynamic project conditions, such as changes in user demand or recent development focus areas [4]. As a result, traditional prioritization techniques may delay the resolution of high-impact bugs, compromising the software’s stability and reliability.

2.3. Machine Learning for Bug Prioritization

With the growing adoption of machine learning (ML) in software engineering, researchers and practitioners are exploring ML-based methods to automate and enhance bug prioritization. Machine learning algorithms can analyze patterns in historical bug data, learn from labeled instances, and predict the priority of new bugs based on their similarity to past issues. This approach allows for data-driven prioritization, reducing human dependency and enabling faster, more consistent decision-making across projects [5]. Previous studies have applied various models, including support vector machines, decision trees, and neural networks, to improve bug prioritization [6].

Transformer-based models, such as BERT and its variants, have shown significant promise. **CodeBERT**, in particular, is pre-trained on a combination of programming and natural language data [7], making it potentially well-suited for analyzing technical issue reports containing code snippets or specific jargon. Standard **BERT** models provide strong general language understanding capabilities. Furthermore, large language models (LLMs) like **GPT-3.5** (and its variants like Airoboros) offer advanced text generation and understanding capabilities that can be leveraged through techniques like prompt engineering for classification tasks. This study compares these different transformer approaches to assess their effectiveness for bug prioritization on GitHub.

3. Proposed Approach

This section outlines the methodology used for prioritizing GitHub issues, including dataset preparation, feature engineering, model selection and configuration, and the evaluation strategy.

3.1. Approach Overview

Our methodology involves three main components:

- **Dataset:** We utilize the GIRT-Data dataset, which includes comprehensive information on over 100,000 GitHub repositories, focusing on their activity and

associated issue reports [8]. This provides a large-scale foundation for analyzing bug prioritization patterns.

- **Model Selection:** We compare the effectiveness of several transformer-based models for the classification task: CodeBERT, standard BERT, and a GPT-3.5-like model. CodeBERT (microsoft/codebert-base) is chosen for its pre-training on both programming and natural language data [7]. Standard BERT (bert-base-uncased) serves as a strong baseline general-purpose language model. The aioboros-gpt-3.5-turbo-100k-7b model is included to evaluate the capabilities of large-scale generative models on this task using prompt engineering. We fine-tuned CodeBERT and BERT to distinguish between high and low-priority issues based on features derived from the dataset.
- **Evaluation Strategy:** Model performance is evaluated using standard classification metrics: precision, recall, F1-score, and accuracy. For CodeBERT, decision threshold adjustments are explored to optimize the balance between precision and recall, addressing the practical need to potentially prioritize one over the other [9], [10].

3.2. Approach Details

3.2.1. Dataset Preparation. The GIRT-Data required several preprocessing steps. Incomplete records were removed, and text fields (like issue descriptions from IRTs) were standardized for consistency. Relevant features concerning repository activity, popularity (stargazers, forks), and issue reporting patterns were extracted to provide context for the models. The dataset was filtered to include only Java repositories with available IRTs. A priority label (high/low) was assigned based on issue activity, using the median issue activity as a threshold (`issue_activity > 50` in our filtered dataset), providing a data-driven split into potentially higher and lower engagement issues [11], [12]. Rows with missing tokenized text or labels were dropped.

3.2.2. Feature Engineering. To enrich the input data beyond the raw text of issue templates, we engineered features from repository characteristics and Issue Report Templates (IRT):

- **Repository-Level Features:** Metrics indicating repository popularity (e.g., stargazer count), commit frequency, recent activity, number of contributors, and issue activity (total open/closed issues) were included. These potentially reflect the significance or urgency of issues within active or widely used repositories [11]–[14]. Repository age was also considered.
- **IRT-Specific Features:** Features extracted from issue templates included the length of the template and the number of structured sections (e.g., headlines

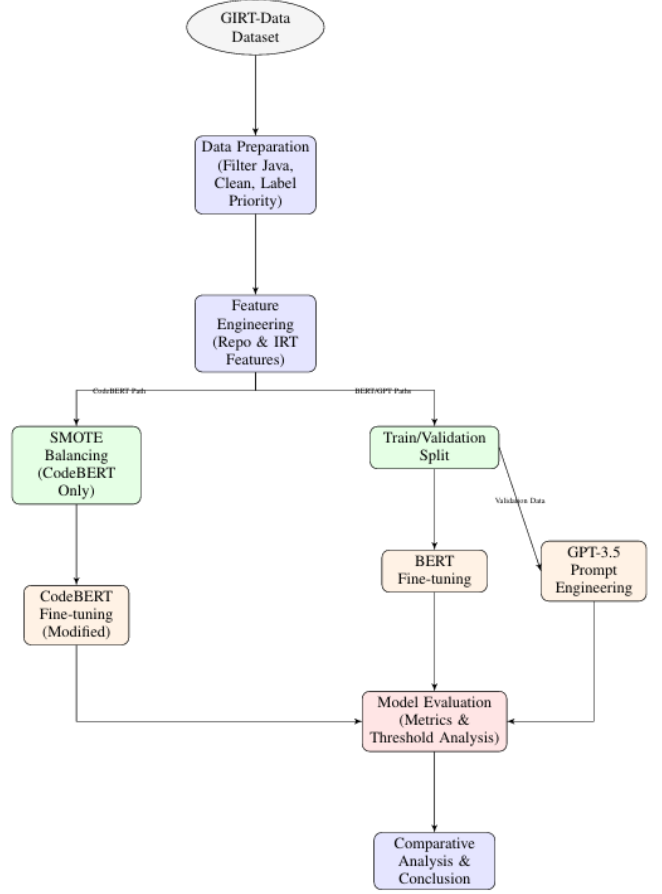


Figure 1. Overview of the research methodology

identified in Markdown templates). The presence or absence of an IRT was also used as a feature. These aim to capture the level of detail or structure in the issue reporting process [11], [12], [15], [16].

Features derived from repository characteristics (stargazers_count, commits_count, contributors_count, open_issues_countv2, closed_issues_countv2, pushed_at, created_at) and IRT characteristics (IRT_raw_markdown, IRT_raw_yaml, headlines) were used [11], [12].

TABLE 1. SUMMARY OF ENGINEERED FEATURES

Category	Feature / Description
Repository	Stargazer Count
	Commit Frequency
	Recent Activity (Pushed $\leq$ 6 mo.)
	Contributor Count
	Issue Activity (Open+Closed)
	Repository Age (Days)
IRT	Has IRT (Boolean)
	IRT Length (Chars)
	Headline Count (Markdown)

3.2.3. Model Selection and Configuration.

- **CodeBERT:** We used the `microsoft/codebert-base` model and its associated `RobertaTokenizer`. It was fine-tuned for binary classification (high/low priority). The data was balanced using SMOTE before splitting [11]. Initial hyperparameters included a learning rate of $5e-5$ and a batch size of 16. To potentially improve performance and handle class imbalance, modifications including freezing the initial 6 encoder layers and applying calculated class weights during loss computation were explored, trained over 5 epochs with early stopping [11], [17]. Evaluation was performed on the validation set derived from the SMOTE-balanced data.
- **BERT:** A standard BERT model, specifically `bert-base-uncased`, was fine-tuned for the same binary classification task using the corresponding `BertTokenizer` and the same text features (`tokenized_body`) as CodeBERT [12]. **SMOTE was not applied** to the data for this model configuration. It was trained for 3 epochs with a batch size of 16 and the default learning rate ($5e-5$). No explicit layer freezing or class weights were applied [12]. Evaluation was performed on the validation set derived from the original, unbalanced data.
- **GPT-3.5 (Airoboros):** The `airoboros-gpt-3.5-turbo-100k-7b` model was utilized via **prompt engineering**. A **few-shot prompting** approach was employed. The prompt included a brief instruction defining the task (classifying issues as high, medium, or low priority), followed by a few examples demonstrating the desired input-output format (Issue text + Priority label). The actual input issue text (`IRT_raw_markdown`) was then provided, along with selected engineered features (`issue_activity`, `stargazers_count`, `repo_age_days`) formatted readably within the prompt to provide additional context. OpenAI's `tiktoken` tokenizer was used to manage input length. The model was then prompted to predict the priority for the given input issue. Evaluation was performed on the same validation set derived from the **original, unbalanced data** (consistent with the BERT evaluation).

3.2.4. Threshold Adjustment (CodeBERT). For the CodeBERT model, recognizing the common trade-off in classification tasks, we experimented with the decision threshold used to convert predicted probabilities into class labels (high/low priority). We evaluated the default threshold of 0.5 and an adjusted threshold of 0.55 to observe the impact on precision and recall, aiming to find a balance suitable for practical deployment scenarios where minimizing false positives or false negatives might be prioritized [9], [11].

4. Evaluations

This section details the experimental setup used to evaluate the different models for bug issue prioritization and presents the results based on the research questions.

4.1. Research Questions

To guide our evaluation, we formulated the following research questions:

- RQ1:** How effective are different transformer-based models (CodeBERT, BERT, GPT-3.5) in prioritizing high-priority GitHub bug issues based on accuracy, precision, recall, and F1-score?
- RQ2:** How do threshold adjustments impact the balance between precision and recall for the CodeBERT model?
- RQ3:** What are the practical implications of using these models in real-world bug prioritization tasks?

4.2. Evaluations Setup

4.2.1. Experiment Design. To evaluate the performance of the models in prioritizing bug issues, we utilized the pre-processed GIRT-Data dataset, focusing on Java repositories with available Issue Report Templates (IRTs). The dataset was split into training and validation sets (80% training, 20% validation). For CodeBERT fine-tuning, the training data was balanced using SMOTE prior to splitting; for BERT fine-tuning and GPT-3.5 prompt engineering evaluation, the original data distribution was used. Models were trained or prompted as described in Section 3.2.3.

4.2.2. Metrics Used. We used the following standard metrics to assess model performance:

- **Accuracy:** Measures the overall proportion of correctly classified instances.
- **Precision:** Represents the ratio of true positives (correctly identified high-priority issues) to the total predicted positives. High precision indicates fewer false alarms.
- **Recall:** Reflects the ratio of true positives to the total actual positives. High recall indicates the model's ability to identify most of the actual high-priority issues.
- **F1-score:** The harmonic mean of precision and recall, providing a balanced measure, particularly useful for imbalanced datasets.

4.3. Evaluation Results

This section presents the quantitative results comparing the different models and discusses qualitative observations.

4.3.1. Comparative Quantitative Results. We evaluated four different model configurations: the original fine-tuned CodeBERT (using an adjusted threshold of 0.55 for balanced performance), CodeBERT with modifications (using class weights, frozen layers, SMOTE, and a threshold of 0.5 for high recall), a standard fine-tuned BERT model (trained on original data), and a GPT-3.5-like model (airoboros-gpt-3.5-turbo-100k-7b) using prompt engineering (evaluated on original data). The performance metrics on the respective validation sets are summarized in Table 2 and visualized in Figure 2.

TABLE 2. PERFORMANCE COMPARISON OF PRIORITIZATION MODELS

Model	Acc.	Prec.	Recall	F1	Eval Data
CodeBERT (Th=0.55)	0.5958	0.5818	0.6480	0.6131	SMOTE Bal.
CodeBERT (Mod., Th=0.5)	0.6173	0.6173	1.0000	0.7634	SMOTE Bal.
BERT Model	0.6720	0.6949	0.8353	0.7586	Original
GPT-3.5	0.7270	0.6769	0.8753	0.7566	Original

Note: "CodeBERT" corresponds to the setup evaluated with threshold=0.55 in main.ipynb. "CodeBERT (Mod., Thresh=0.5)" corresponds to the setup evaluated with threshold=0.5, incorporating class weights, layer freezing, and SMOTE.

The results indicate that the **GPT-3.5 (Airoboros) model using prompt engineering achieved the highest accuracy (72.70%)** when evaluated on the original data distribution. The **modified CodeBERT model achieved perfect recall (1.0000)** on the SMOTE-balanced validation set, meaning it identified all actual high-priority issues in that specific evaluation context, resulting in the highest F1-score (76.34%). The standard **BERT model also performed strongly** on the original data distribution, achieving a good balance between precision (69.49%) and recall (83.53%), resulting in an F1-score (75.86%) very close to the modified CodeBERT and GPT-3.5. The original CodeBERT configuration (with threshold 0.55, evaluated on balanced data) showed lower performance across all metrics compared to the other models tested.

4.3.2. Qualitative Results and Threshold Impact.

- **Model Comparison:** The superior performance of the standard BERT and the prompt-engineered GPT-3.5 model compared to the initial CodeBERT setup (when evaluated on respective datasets) suggests that either the broader pre-training of standard BERT or the generative capabilities and scale of GPT-3.5 might be more advantageous for capturing the signals related to issue priority in this dataset than CodeBERT’s specific code+NL pre-training alone. The modified CodeBERT’s perfect recall highlights the effectiveness of threshold tuning and SMOTE for specific objectives like ensuring no high-priority issue is missed in a balanced evaluation scenario, though this often comes at the cost of lower precision. The strong performance of BERT and GPT-3.5 on the original (unbalanced) data suggests they

generalize well even without explicit balancing techniques like SMOTE in this case.

- **Threshold Impact (CodeBERT):** As observed in the CodeBERT experiments, adjusting the decision threshold significantly impacts the precision-recall trade-off (RQ2). A threshold of 0.5 (CodeBERT modified) maximized recall, suitable when missing critical issues is unacceptable. Increasing the threshold to 0.55 improved precision by reducing false positives but lowered recall. This demonstrates the flexibility needed for practical application (RQ3), allowing teams to tune the model based on whether they prioritize minimizing missed critical bugs (high recall) or reducing the noise from false alarms (high precision). The optimal balance would depend on the specific team’s workflow and tolerance for false positives versus false negatives.

5. Threats to Validity

Several factors could potentially affect the validity of our findings. We discuss these threats concerning internal, external, reliability, and construct validity.

5.1. Internal Validity

Internal validity relates to the confidence that the observed results are due to the experimental manipulations rather than confounding factors. Potential threats include:

- **Feature Selection:** The specific choice of engineered features (e.g., stargazer count, commit frequency, IRT length) might influence model performance. While selected based on domain knowledge and prior work, other unobserved repository or issue characteristics could act as confounders. We attempted to mitigate this by using a diverse set of features, but the optimal set might differ.
- **Data Preprocessing:** Variations in data cleaning, such as handling missing values (though we filtered incomplete data) or standardizing text formats across different IRT types (Markdown vs. YAML), could introduce subtle biases affecting model training.
- **Priority Label Definition:** Our priority label was derived programmatically based on issue activity (`issue_activity > 50`). This serves as a proxy for priority but may not perfectly align with the actual priority assigned by developers in all cases, potentially affecting the model’s learned associations.
- **SMOTE Application:** Applying SMOTE only to the CodeBERT training data introduces a methodological difference compared to BERT and GPT-3.5 evaluations. While this allowed CodeBERT to achieve high recall on its specific balanced validation set, it makes direct comparison slightly less straightforward, although evaluating BERT/GPT-3.5 on the original distribution provides a realistic baseline.

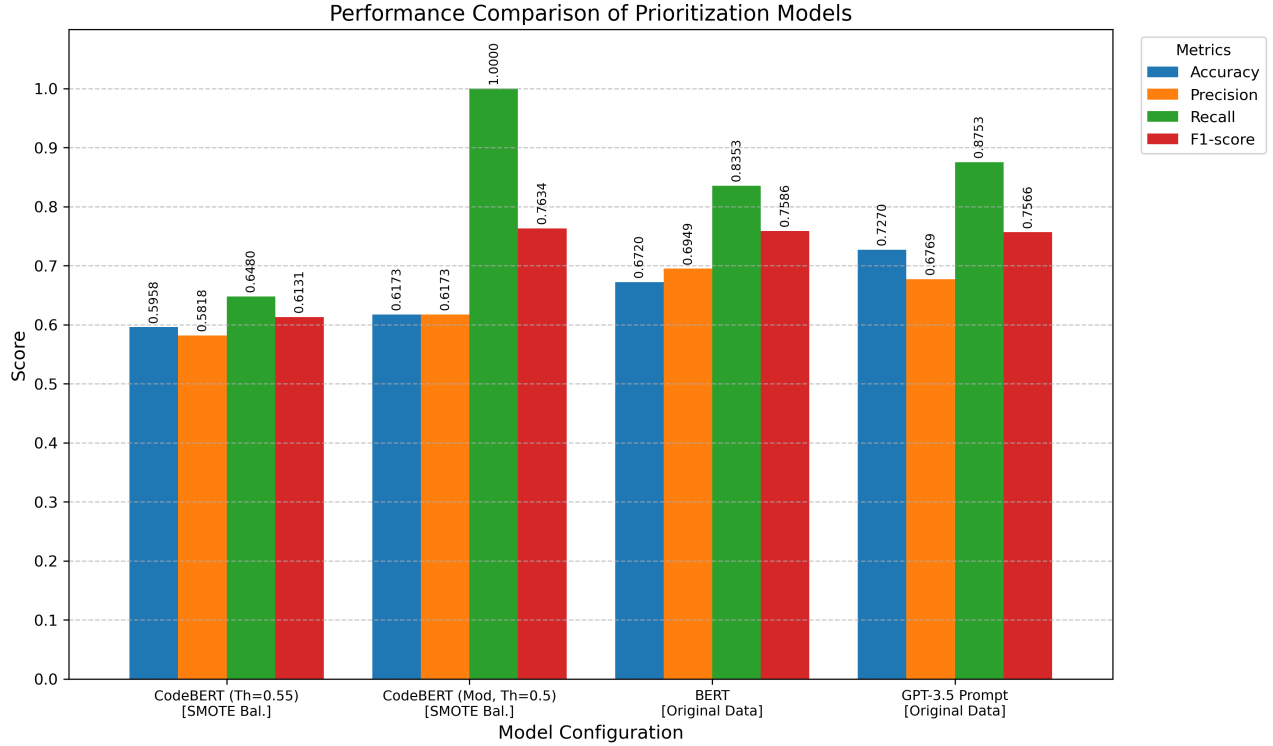


Figure 2. Comparison of performance metrics across different model configurations. (CodeBERT (Th=0.55) and CodeBERT (Mod., Th=0.5) evaluated on SMOTE balanced data; BERT Model and GPT-3.5 Prompt evaluated on original data.)

We aimed for consistency in preprocessing where possible and documented our model configurations, but these factors represent potential internal threats [13], [17].

5.2. External Validity

External validity concerns the generalizability of our findings to other contexts. Threats include:

- **Dataset Scope:** Our experiments rely on the GIRT-Data dataset, primarily containing open-source GitHub repositories. The findings might not directly generalize to proprietary software projects, different platforms (e.g., Jira, Bugzilla), or repositories using different development practices or issue tracking conventions.
- **Language Focus:** We filtered the dataset to focus only on Java repositories. The effectiveness of the models and features might differ significantly for projects written in other programming languages (e.g., Python, JavaScript, C++) which may have different common bug types or reporting styles.
- **Temporal Validity:** The dataset captures a specific time window. Software development practices, GitHub features (like IRTs), and the nature of bugs evolve, potentially limiting the long-term applicability of models trained on older data without retraining.

Further studies applying these models to diverse datasets across different languages, project types, and time periods are needed to strengthen external validity [14], [15].

5.3. Reliability

Reliability refers to the reproducibility of our study. We took steps to enhance reliability:

- **Code Availability:** The code notebooks detailing data processing, feature engineering, and model training/evaluation are documented [11], [12].
- **Methodology Description:** We have detailed the dataset, preprocessing steps, feature engineering, model configurations, and evaluation metrics in Sections 3 and 4.
- **Standard Libraries/Models:** We utilized publicly available models (CodeBERT, BERT) and libraries (Hugging Face Transformers, scikit-learn, etc.) with specified versions where possible.

However, inherent randomness in ML training (e.g., weight initialization, data shuffling) and potential minor differences in software environments might lead to slight variations in results upon replication [9].

5.4. Construct Validity

Construct validity assesses whether the chosen metrics accurately measure the concept being studied (i.e., effective

bug prioritization).

- **Metrics Choice:** We used standard classification metrics (Accuracy, Precision, Recall, F1-score). While informative, they might not fully capture the practical utility for developers. For instance, the cost of misclassifying a high-priority bug as low might be much greater than the reverse. Metrics like weighted accuracy or cost-sensitive evaluation could provide further insights.
- **Proxy for Priority:** As mentioned under internal validity, using issue activity as a proxy for ground-truth priority is a potential threat. While a reasonable heuristic, it simplifies the complex decision-making process developers use for actual prioritization.
- **Model Evaluation Context:** Evaluating CodeBERT on SMOTE-balanced data versus BERT/GPT-3.5 on original data affects the direct comparability for some metrics like recall, although it reflects different potential operational goals (maximize recall vs. performance on natural distribution).

Future work could incorporate developer surveys or user studies to evaluate how well the model’s predictions align with human expert judgment, providing a more comprehensive assessment of construct validity [7], [10].

6. Related Work

Automating tasks within the software development life-cycle, particularly bug report management, has been an active area of research. This section reviews existing work relevant to our study, focusing on machine learning approaches for bug prioritization, automated triage systems, and feature engineering techniques.

6.1. Machine Learning Approaches for Prioritization

Machine learning (ML) has been increasingly applied to enhance bug prioritization, aiming to improve efficiency over manual methods. Early work often focused on traditional ML algorithms. For example, Lamkanfi et al. [6], [10] used Naive Bayes and SVM with TF-IDF features derived from bug report text to predict severity. Kim et al. [5] explored classifying software changes as clean or buggy using text mining and complexity metrics. More recently, transformer-based models have gained traction due to their superior performance in natural language understanding. Gomes et al. [18] compared BERT-based feature extraction against TF-IDF for predicting long-lived bugs in open-source projects, finding BERT features combined with classifiers like SVM and Random Forest more effective. Ahmed et al. [19] also utilized NLP and supervised learning (including Random Forest) for bug categorization and prioritization, employing SMOTE to handle class imbalance, similar to our approach for CodeBERT. Our work builds on these by specifically comparing a code-aware model

(CodeBERT), a standard BERT, and an LLM (GPT-3.5 via prompting) within the context of GitHub issues utilizing IRTs and repository metadata. Broader studies on ML for task prioritization, such as the systematic literature review by Bugayenko et al. [20] and the taxonomy for test case prioritization by Ramírez et al. [21], provide context on evaluation metrics and relevant information attributes, informing our choice of metrics and features.

6.2. Automated Triage and Analysis Systems

Beyond prioritization, ML is used for broader bug triage, including assigning developers or identifying duplicate reports. While not the direct focus of our study, these systems often share similar challenges and techniques. Izadi et al. [22] proposed a two-stage method using RoBERTa to predict the objective and priority of issue reports, incorporating feature engineering and sentiment analysis. Jahanshahi et al. [23] introduced ADPTriage, using approximate dynamic programming for adaptive bug assignment optimization. The structure and content of bug reports themselves have also been studied. Li et al. [16] investigated the use and structure of bug report templates (IRTs) on GitHub, demonstrating their value for structured reporting and proposing methods for automatic template generation, which aligns with our use of IRT content and structure as features. Sitohang et al. [24] applied models like non-linear SVM for severity classification, contributing to the body of work on classifying issue characteristics. Our study leverages insights from these areas by using IRT content and repository metadata as input for prioritization models.

6.3. Feature Engineering and Selection

The choice of features significantly impacts the performance of ML models in software engineering tasks. Textual features derived using techniques like TF-IDF, word embeddings, or transformer-based embeddings (like BERT) are common [18], [19]. Metadata features, such as product/component information, submitter details, or historical data, have also been explored [3], [17]. Recent work, including Izadi et al. [22], emphasizes combining advanced textual representations (from fine-tuned transformers) with carefully engineered metadata features. Rathnayake et al. [25] explored CNNs for priority prediction, suggesting alternative architectures for processing report data. Colavito et al. [26] investigated using GPT-like models for zero-shot issue labeling, showcasing the potential of LLMs to reduce reliance on labeled data, which contrasts with our few-shot prompting approach that still utilized examples. Our work contributes by explicitly incorporating features related to repository activity (commits, stars, age) and IRT structure (length, headlines) alongside the textual content processed by different transformer models.

6.4. Positioning Our Work

While existing studies have applied various ML techniques, including transformers like BERT [18] and

RoBERTa [22], for bug analysis and prioritization, our work provides a direct comparison between a code-specialized model (CodeBERT), a general-purpose model (BERT), and an LLM-based prompt engineering approach (GPT-3.5/Airoboros) specifically for prioritizing GitHub issues using IRT text and repository context features. We also explicitly analyze the impact of techniques like SMOTE and threshold tuning in this comparative context. By evaluating these distinct approaches, we aim to provide clearer insights into the trade-offs and effectiveness of different state-of-the-art models for this specific, practical task.

7. Discussion

This study compared the effectiveness of CodeBERT, standard BERT, and a GPT-3.5-like model (Airoboros via prompt engineering) for prioritizing GitHub bug issues based on issue report templates and repository metadata. Our evaluation yielded several key findings regarding the performance and applicability of these models.

7.1. Model Performance Comparison (RQ1)

The results presented in Table 2 show notable differences in performance across the tested models. While the modified CodeBERT achieved the highest F1-score (0.7634) and perfect recall on the SMOTE-balanced validation set, the prompt-engineered GPT-3.5 (Airoboros) model achieved the highest accuracy (0.7270) on the original, unbalanced validation set. Standard BERT also demonstrated strong performance on the original data, with an F1-score (0.7586) nearly matching the modified CodeBERT and GPT-3.5, coupled with high precision (0.6949) and recall (0.8353).

Interestingly, the standard CodeBERT configuration (threshold = 0.55, evaluated on balanced data) did not outperform the standard BERT model evaluated on the original data distribution. This suggests that for this specific task and dataset, CodeBERT’s specialized pre-training on code might not have provided a distinct advantage over BERT’s general language understanding capabilities when applied to the IRT text and engineered features. It is possible that the textual content within the IRTs, even for Java repositories, relied more heavily on natural language descriptions of problems rather than complex code snippets where CodeBERT might excel. Furthermore, the features engineered from repository metadata might be more effectively interpreted by models with broader world knowledge or general language pre-training compared to code-specific pre-training.

The strong performance of the standard BERT model, even without SMOTE balancing, indicates its robustness in handling the inherent class imbalance present in the original data for this task. Furthermore, the high accuracy achieved by the GPT-3.5 (Airoboros) model using few-shot prompt engineering is noteworthy. This approach, which combined raw text with key engineered features (`issue_activity`, `stargazers_count`, `repo_age_days`) directly in the prompt, likely benefited from the LLM’s extensive world knowledge and its ability to leverage context effectively,

even with limited examples. This suggests that prompt-based approaches with LLMs are a viable, and potentially highly effective, alternative to fine-tuning specialized models for bug prioritization.

7.2. Impact of SMOTE and Threshold Tuning (RQ2)

The experiments with CodeBERT highlighted the significant impact of methodological choices like data balancing and threshold adjustment. Applying SMOTE and using a threshold of 0.5 allowed the modified CodeBERT model to achieve perfect recall on the balanced validation set, effectively identifying all high-priority issues in that context. However, this came at the cost of precision, as indicated by the F1-score not being proportionally higher than accuracy compared to other models. Evaluating BERT and GPT-3.5 on the original, unbalanced data provided perhaps a more realistic assessment of performance, where both models achieved strong results without explicit balancing. This raises questions about the necessity and potential impact of SMOTE – while it helped CodeBERT maximize recall in a balanced setting, the other models performed well on the natural distribution.

The threshold tuning for CodeBERT explicitly demonstrated the precision-recall trade-off. Adjusting the threshold from 0.5 to 0.55 increased precision but decreased recall and F1-score, confirming that the optimal threshold depends heavily on the specific requirements of the deployment scenario (RQ3).

7.3. Practical Implications (RQ3)

The choice of the “best” model depends on the practical goals of the development team.

- If the primary goal is to ensure absolutely no high-priority bugs are missed (maximizing recall), the modified CodeBERT approach (with SMOTE and threshold=0.5) appears most effective based on its performance on the balanced dataset, although teams must be prepared to handle a higher number of false positives.
- If the goal is highest overall accuracy in predicting priority on naturally distributed data, the GPT-3.5 (Airoboros) prompt engineering approach showed the best results in our experiments.
- If a balance between precision and recall on the original data distribution is desired, standard BERT offers strong performance with the advantage of being a well-understood fine-tuning process compared to prompt engineering LLMs.

Furthermore, practical considerations include computational resources and ease of implementation. Fine-tuning models like BERT and CodeBERT requires significant computational resources for training but results in a self-contained model. Prompt engineering with large models like

Airoboros/GPT-3.5 might require less specific ML infrastructure but could involve API costs and dependencies, and careful prompt design is crucial.

Our findings suggest that while specialized models like CodeBERT are relevant, standard BERT and prompt-engineered LLMs are highly competitive and potentially superior alternatives for prioritizing GitHub issues based on IRTs and repository context, especially when considering performance on unbalanced, real-world data distributions.

8. Limitations

While this study provides valuable insights into comparing different transformer models for bug prioritization, certain limitations should be acknowledged.

8.1. Dataset Limitations

The model evaluation was performed using data derived from the GIRT-Data dataset, which, while extensive, primarily contains information from open-source GitHub repositories. This focus might limit the generalizability of our findings to proprietary software development environments, which often have different workflows, user bases, issue complexity, terminology, and reporting practices. Furthermore, the filtering for Java repositories means the conclusions might not directly apply to projects using other programming languages, where bug characteristics and reporting styles could vary significantly.

8.2. Dependence on Feature Quality and Selection

The performance of the fine-tuned models (CodeBERT, BERT) and the prompt-engineered model (Airoboros) relies on the quality and relevance of the engineered features derived from repository metadata and IRT characteristics. While features like popularity metrics, issue activity, and IRT length provide useful context, their effectiveness might vary across different projects or domains. Some potentially influential factors, such as the expertise level of the bug reporter, the specific code modules affected (if identifiable), or dynamic project priorities, were not included due to data limitations or complexity in extraction. The reliance on the current feature set might restrict the models' capability to fully capture all nuances influencing bug priority.

8.3. Model Complexity and Computational Requirements

Fine-tuning transformer models like CodeBERT and BERT is computationally intensive, requiring significant GPU resources and time. This poses a barrier for teams or researchers with limited computational budgets. Prompt engineering with large models like Airoboros might reduce the need for local training infrastructure but can incur API costs and depends on the availability and reliability of external services. The complexity also impacts deployment and maintenance overhead.

8.4. Challenges in Threshold Tuning and Prompt Engineering

While threshold adjustments (demonstrated for CodeBERT) offer flexibility in balancing precision and recall, selecting the optimal threshold requires careful validation and depends on specific project needs, which can be challenging to determine universally. Similarly, the effectiveness of the prompt-engineered GPT-3.5 model is highly sensitive to the prompt design (instructions, examples, feature formatting). Finding the optimal prompt often requires significant experimentation and iteration, which was not exhaustively explored in this study.

8.5. Reliance on Static Training Data and Evaluation

The fine-tuned models currently rely on static training data. Software projects evolve, leading to changes in bug types, reporting patterns, and team priorities (concept drift). Models trained on historical data may see performance degrade over time without periodic retraining or adaptation mechanisms like online or active learning. Furthermore, the evaluation was conducted in a controlled offline setting using a fixed validation split, which may not fully reflect the dynamic nature and complexities of integrating such a tool into a live bug triaging workflow.

8.6. Interpretability Challenges

Transformer models, including BERT, CodeBERT, and large models like GPT-3.5, are often considered "black boxes". Understanding *why* a model assigned a particular priority to a bug report can be difficult. This lack of interpretability can be a barrier to adoption, especially in contexts where developers need to trust and understand the rationale behind automated decisions. While not explored here, applying explainable AI (XAI) techniques could mitigate this limitation in future work.

8.7. Proxy for Ground Truth Priority

Our study used issue activity as a proxy to define the binary priority labels (high/low). While this provides an objective measure based on data, it may not perfectly mirror the nuanced priority assigned by human developers, which can involve factors like severity, user impact, development effort, and strategic goals. Using this proxy introduces a potential construct validity limitation, as the models are optimized for the proxy label rather than the potentially more complex human-assigned priority.

9. Conclusions and Future Work

9.1. Summary of Findings

This study investigated the effectiveness of different transformer-based models for prioritizing GitHub bug issue

reports, specifically comparing fine-tuned CodeBERT and standard BERT models against a prompt-engineered GPT-3.5-like model (Airoboros). Using a dataset derived from GIRT-Data focusing on Java repositories, we evaluated these approaches based on their ability to classify issues into high or low priority categories using IRT text and engineered repository/IRT features.

Our comparative analysis revealed significant findings. While the specialized CodeBERT model, particularly when modified with techniques like SMOTE and threshold tuning, demonstrated its capability by achieving perfect recall (1.0000) for high-priority issues on balanced data and the highest F1-score (76.34%), it was outperformed in overall accuracy on unbalanced data. Standard BERT and the prompt-engineered GPT-3.5 model demonstrated superior or comparable performance on the original, unbalanced data distribution, suggesting robustness to natural class imbalance. The GPT-3.5 approach yielded the highest accuracy (72.70%), highlighting the potential of LLMs in this domain, while standard BERT provided a strong balance of precision (0.6949) and recall (0.8353), achieving a competitive F1-score (75.86%). These results suggest that general language understanding (BERT) or the contextual reasoning capabilities leveraged through prompt engineering (GPT-3.5) can be highly effective for this task, potentially outweighing the benefits of CodeBERT's code-specific pre-training for the type of data analyzed here. Furthermore, the study confirmed the significant impact of data balancing techniques (SMOTE) and decision threshold adjustments on model performance, particularly concerning the precision-recall trade-off, which is crucial for practical application.

9.2. Future Directions

Based on our findings and identified limitations, several promising avenues for future research emerge to build upon this work:

- **Exploring Alternative Models and Architectures:** Further investigation could involve comparing performance with other transformer architectures (e.g., RoBERTa, DistilBERT, XLNet) or more recent and capable LLMs. Systematically fine-tuning larger models like LLaMA variants specifically for this classification task, assessing the trade-offs between performance gains and computational cost, could yield valuable insights compared to prompt engineering alone.
- **Enhancing Feature Engineering:** Incorporating a richer set of features could further improve model performance and provide deeper context. This includes analyzing the sentiment expressed within issue comments, quantifying developer engagement levels (e.g., number of participants, response times), considering the reporter's historical contribution or experience level, and utilizing more fine-grained code metrics derived directly from associated commits or code snippets if available.

- **Investigating Prompt Engineering Strategies:** For LLM-based approaches, a systematic exploration of different prompt engineering techniques is warranted. This involves comparing zero-shot vs. few-shot performance with varying numbers and quality of examples, optimizing the prompt structure and instructions, and rigorously assessing the impact of including different combinations or formats of engineered features within the prompt on both accuracy and consistency.
- **Cross-Project and Cross-Language Generalization:** Evaluating the models on a more diverse dataset is crucial for assessing real-world applicability. This includes incorporating repositories from different domains (e.g., web development, data science, commercial software) and various programming languages beyond Java. Training or evaluating multi-language models could significantly broaden the utility of such prioritization tools.
- **Real-World Integration and Evaluation:** Deploying the most promising models (e.g., standard BERT fine-tuning or the GPT-3.5 prompt approach) within a real-world bug triaging system via pilot studies or A/B testing would provide invaluable insights. This allows for measuring practical utility, scalability, latency, and integration challenges within actual development workflows, incorporating direct feedback from developers for iterative refinement.
- **Interpretability and Explainability:** Addressing the "black box" nature of complex transformer models remains important for building trust and facilitating adoption. Future work could apply model-agnostic explainability techniques (like SHAP or LIME) or explore model-specific attention mechanisms to understand which features or parts of the input text most influence the prioritization decisions, thereby increasing transparency.
- **Active Learning and Continuous Adaptation:** To combat potential concept drift and maintain model performance as software projects evolve and bug reporting patterns change, incorporating active learning or online learning strategies could be beneficial. These approaches could allow the models to adapt over time by intelligently querying for human feedback on the most uncertain predictions, reducing the need for large-scale manual re-labeling efforts.

9.3. Conclusion

In conclusion, this study provides a valuable comparative analysis of CodeBERT, BERT, and a GPT-3.5-like model for the critical task of prioritizing GitHub bug issues. Our results demonstrate that while specialized models like CodeBERT offer specific advantages, particularly when tailored for high recall, standard BERT and prompt-engineered LLMs present highly effective and competitive alternatives, especially when evaluated on realistic, unbalanced data. The findings underscore that the optimal approach depends heav-

ily on specific project goals regarding the balance between accuracy and recall, acceptable false positive rates, available computational resources, and implementation complexity. This research contributes empirical evidence to the understanding of applying diverse advanced machine learning models to automate and improve bug prioritization, offering insights that can aid software development teams in managing issue backlogs more efficiently and effectively. Continued research, particularly focusing on real-world validation, enhanced feature engineering, and model interpretability, will be essential to further advance the state-of-the-art and practical utility of automated bug prioritization in software engineering.

References

- [1] P. J. Guo, T. Zimmermann, N. Nagappan, and B. Murphy, "Characterizing and predicting which bugs get fixed: An empirical study of microsoft windows," *IEEE Transactions on Software Engineering*, vol. 32, no. 6, pp. 654–669, 2010.
- [2] F. Zhang, S. Yang, and A. Mockus, "A study of the uniqueness of signature change as a defect indicator," *IEEE Transactions on Software Engineering*, vol. 42, no. 3, pp. 268–283, 2016.
- [3] N. Bettenburg, S. Just, A. Schröter, C. Weiss, R. Premraj, and T. Zimmermann, "What makes a good bug report?," in *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pp. 308–318, 2008.
- [4] K. Czarnecki and J. Dietrich, "Characterizing priority bugs," *Journal of Software: Evolution and Process*, vol. 25, no. 5, pp. 465–494, 2013.
- [5] S. Kim, E. J. Whitehead, Jr., and Y. Zhang, "Classifying software changes: Clean or buggy?," *IEEE Transactions on Software Engineering*, vol. 34, no. 2, pp. 181–196, 2008.
- [6] A. Lamkanfi, S. Demeyer, E. Giger, and B. Goethals, "Predicting the severity of a reported bug," in *Proceedings of the 7th IEEE Working Conference on Mining Software Repositories (MSR)*, pp. 1–10, 2010.
- [7] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "Codebert: A pre-trained model for programming and natural languages," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1536–1547, 2020.
- [8] N. Nikeghbal, A. H. Kargaran, and A. Heydarnoori, "Girt-model: Automated generation of issue report templates," 2024.
- [9] S. Kim, E. J. Whitehead, Jr., and Y. Zhang, "Classifying software changes: Clean or buggy?," in *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pp. 308–318, 2008.
- [10] A. Lamkanfi, S. Demeyer, E. Giger, and B. Goethals, "Predicting the severity of a reported bug," in *Proceedings of the 7th IEEE Working Conference on Mining Software Repositories (MSR)*, pp. 1–10, 2010.
- [11] S. Khan, "Codebert implementation for bug prioritization," 2024. Jupyter Notebook, Supplementary Material.
- [12] S. Khan, "Bert implementation for bug prioritization," 2025. Jupyter Notebook, Supplementary Material.
- [13] P. J. Guo, T. Zimmermann, N. Nagappan, and B. Murphy, "Characterizing and predicting which bugs get fixed: An empirical study of microsoft windows," *IEEE Transactions on Software Engineering*, vol. 32, no. 6, pp. 654–669, 2010.
- [14] F. Zhang, S. Yang, and A. Mockus, "A study of the uniqueness of signature change as a defect indicator," *IEEE Transactions on Software Engineering*, vol. 42, no. 3, pp. 268–283, 2016.
- [15] K. Czarnecki and J. Dietrich, "Characterizing priority bugs," *Journal of Software: Evolution and Process*, vol. 25, no. 5, pp. 465–494, 2013.
- [16] H. Li, M. Yan, W. Sun, X. Liu, and Y. Wu, "A first look at bug report templates on github," *Journal of Systems and Software*, vol. 202, p. 111709, 2023.
- [17] N. Bettenburg, S. Just, A. Schröter, C. Weiss, R. Premraj, and T. Zimmermann, "What makes a good bug report?," in *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pp. 308–318, 2008.
- [18] L. Gomes, R. da Silva Torres, and M. L. Cortes, "Bert- and tf-idf-based feature extraction for long-lived bug prediction in floss: A comparative study," *Information and Software Technology*, vol. 160, p. 107217, 2023.
- [19] H. A. Ahmed, N. Z. Bawany, and J. A. Shamsi, "Capbug - a framework for automatic bug categorization and prioritization using nlp and machine learning algorithms," *IEEE Access*, vol. 9, pp. 50496–50512, 2021.
- [20] Y. Bugayenko, A. Bakare, A. Cheverda, M. Farina, A. Kruglov, and Y. Plaksin, "Prioritizing tasks in software development: A systematic literature review," *PLoS ONE*, vol. 18, no. 4, p. e0283838, 2023.
- [21] A. Ramírez, R. Feldt, and J. R. Romero, "A taxonomy of information attributes for test case prioritisation: Applicability, machine learning," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 1, pp. 21:1–21:42, 2023.
- [22] M. Izadi, K. Akbari, and A. Heydarnoori, "Predicting the objective and priority of issue reports in software repositories," *Empirical Software Engineering*, vol. 27, no. 2, p. 50, 2022.
- [23] H. Jahanshahi, M. Cevik, K. Mousavi, and A. Başar, "Adptriage: Approximate dynamic programming for bug triage," *IEEE Transactions on Software Engineering*, vol. 49, no. 11, pp. 1–18, 2023.
- [24] F. Sitohang, F. Sianipar, and M. E. S. Simaremare, "Severity level labeling for software issue data using machine learning techniques," in *2023 29th International Conference on Telecommunications (ICT)*, pp. 1–5, 2023.
- [25] R. M. D. S. Rathnayake, B. T. G. S. Kumara, and E. M. U. W. J. B. Ekanayake, "Cnn-based priority prediction of bug reports," in *2021 International Conference on Decision Aid Sciences and Application (DASA)*, pp. 299–303, 2021.
- [26] G. Colavito, F. Lanubile, N. Novielli, and L. Quaranta, "Leveraging gpt-like llms to automate issue labeling," arXiv preprint arXiv:2401.10431, 2024.